D3.js

Building an Interactive Graphing Calculator

Data-Driven Documents · SVG · Scales · Axes · Animation · Zoom

A lesson in D3.js · Using the calculator project as a running example

What Is D3.js?

CORE CONCEPT

Data-Driven Documents

D3 connects data to HTML and SVG elements.
Instead of drawing visuals by hand, you bind data arrays to elements
— D3 handles the rendering.

SVG Control

Works directly with HTML & SVG elements for pixel-perfect visuals

Data Binding

Connects data arrays to DOM elements declaratively

Custom Viz

More flexible than any pre-built chart library

Ecosystem

Scales, axes, shapes, animation, zoom & interaction built-in

In our project: The calculator creates data points from a math expression, then D3 turns those points into an SVG graph.

📎 **D3 main site:** https://d3js.org/

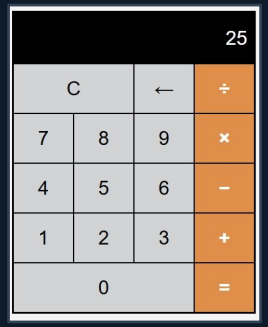
📎 **Getting started:** https://d3js.org/getting-started

The Project: Before & After

BEFORE

Basic Calculator

- HTML, CSS & JavaScript
- One number, one operator, one number
- Simple step-by-step arithmetic only
- No expressions, no graphing



AFTER

Graphing Calculator

- Powered by D3.js + Math.js
- Builds full expression strings
- "=" evaluates number-only expressions
- "Graph" draws expressions with x
- Animated, interactive SVG graph
- Hover, zoom, and pan support

```
Examples:  2+3*4   x^2-4   sin(x)   sqrt(x)
```

D3 Concept: Selections

D3 always starts by selecting a page element.

How It Works

d3.select() grabs an existing HTML or SVG element from the page. Once selected, you can append child elements, set attributes, bind data, or add interactions.

General example:

```
d3.select("#chart")     // select by ID
  .append("svg")       // add child element
  .attr("width", 500);  // set attribute
```

📎 **Selections:** https://d3js.org/d3-selection

In Our Calculator

The SVG graph area is selected first, then axes, paths, and interaction layers are appended inside it.

```
const svg = d3.select(functionGraph);


const chart = svg
  .append("g")
  .attr("transform",
```

Everything else D3 does to the graph flows from this first selection.

📎 **Selecting elements:** https://d3js.org/d3-selection/selecting

D3 Concept: Scales

Scales convert data values into screen positions.

domain([-10, 10])
Math values

→

range([0, innerWidth])
Screen pixels

Mini example:

```
const scale = d3.scaleLinear()
  .domain([0, 100])   // data range
  .range([0, 500]);   // pixel range

console.log(scale(50)); // → 250
```

Y range is reversed — SVG 0 is at the top.

📎 **Linear scales:** https://d3js.org/d3-scale/linear

In Our Calculator

Two scales map the mathematical coordinate system to SVG pixel coordinates:

```
const baseXScale = d3
  .scaleLinear()
  .domain([-10, 10])
  .range([0, innerWidth]);

const baseYScale = d3
  .scaleLinear()
  .domain([-10, 10])
  .range([innerHeight, 0]);
```

D3 Concept: Axes

D3 generates readable axes directly from scales.

X Axis

```
xAxisGroup
  .attr("transform",
    `translate(0, ${yScale(0)})`)
  .call(
    d3.axisBottom(xScale).ticks(5)
  );
```

Key points:

- Axes are placed at y=0 and x=0, not at slide edges
- Grid lines use a hidden axis with long ticks
- tickFormat('') removes grid labels

📎 **Axes:** https://d3js.org/d3-axis

Y Axis & Grid

```
yAxisGroup
  .attr("transform",
    `translate(${xScale(0)}, 0)`)
  .call(
    d3.axisLeft(yScale).ticks(5)
  );


// Grid lines
gridGroup.call(
  d3.axisLeft(yScale)
    .ticks(5)
    .tickSize(-innerWidth)
    .tickFormat("")
```

D3 Concept: Line & Area Generators

D3 turns arrays of data points into SVG path strings.

d3.line() — The Curve

Converts x/y point arrays into an SVG path. The defined() check drops extreme off-screen values.

```
const line = d3.line()
  .defined(p =>
    p.y >= yScale.domain()[0] &&
    p.y <= yScale.domain()[1])
  .x(p => xScale(p.x))
  .y(p => yScale(p.y))
  .curve(d3.curveMonotoneX);

linePath.datum(data).attr("d", line);
```

📎 **d3-shape/line:** https://d3js.org/d3-shape/line

d3.area() — Shaded Fill

Like d3.line(), but fills the region between the function and the x-axis (y0 = 0).

```
const area = d3.area()
  .defined(p =>
    p.y >= yScale.domain()[0] &&
    p.y <= yScale.domain()[1])
  .x(p => xScale(p.x))
  .y0(yScale(0))       // baseline
  .y1(p => yScale(p.y)) // curve
  .curve(d3.curveMonotoneX);
areaPath.datum(data).attr("d", area);
```

📎 **d3-shape/area:** https://d3js.org/d3-shape/area

D3 Concept: Data Binding

D3's core idea: bind data arrays to DOM/SVG elements.

Standalone example — list from data:

```
const fruits = ["Apple", "Banana", "Cherry"];


d3.select("#list")
  .selectAll("li")
  .data(fruits)
  .join("li")
  .text(fruit => fruit);
// Creates one <li> per fruit
```

📎 **d3-selection:** https://d3js.org/d3-selection

In Our Calculator

Each hover point on the graph is a circle created by data binding:

```
chart
  .selectAll(".graph-point")
  .data(hoverPoints)
  .join("circle")
  .attr("cx", p => xScale(p.x))
  .attr("cy", p => yScale(p.y))
```

📎 **Modifying elements:** https://d3js.org/d3-selection/modifying

D3 Concept: Animation & Interaction

STEPS 6–8 OF 8

Transitions

Animate SVG changes smoothly.

```
// Draw-on animation:
const len =
  linePath.node()
    .getTotalLength();

linePath
  .attr("stroke-dasharray",
    `${len} ${len}`)
  .attr("stroke-dashoffset", len)
  .transition()
  .duration(650)
  .ease(d3.easeCubicOut)
  .attr("stroke-dashoffset", 0);
```

📎 **Transitions:** https://d3js.org/d3-transition

Events & Tooltip

Hover, click, mousemove and more.

```
.on("mouseenter",
  (event, point) => {
    graphTooltip
      .hidden = false;
    graphTooltip
      .textContent =
      `x: ${point.x.toFixed(1)},
       y: ${point.y.toFixed(2)}`;
  });

// Crosshair:
crosshairX.attr("x1", x)
  .attr("x2", x);
```

📎 **Events:** https://d3js.org/d3-selection/events

Zoom & Pan

Scroll or drag to explore the graph.

```
const zoom = d3.zoom()
  .scaleExtent([0.8, 8])
  .on("zoom", (event) => {
    const zX =
      event.transform
        .rescaleX(baseXScale);
    const zY =
      event.transform
        .rescaleY(baseYScale);

    renderGraph(zX, zY);
  });
svg.call(zoom);
```

📎 **Zoom behavior:** https://d3js.org/d3-zoom

Supporting Library: Math.js

SUPPORTING ROLE

Math.js supplies the numeric data that D3 needs to draw the graph.

math.evaluate() — "=" Button

Evaluates the expression once. Used when the user presses equals.

```
// Mini example:
math.evaluate("2 + 3 * 4"); // → 14


// In the calculator:
lastAnswer = formatResult(
  math.evaluate(expression, {
    ans: Number(lastAnswer)
  })
);
```

📎 **math.evaluate:** https://mathjs.org/docs/reference/functions/evaluate.html

math.compile() — "Graph" Button

Compiles once, evaluates many times — one per x-value across the graph.

```
// Mini example:
const fn = math.compile("x^2 + 1");
fn.evaluate({ x: 2 }); // → 5
fn.evaluate({ x: 3 }); // → 10
// In the calculator:
const compiled =
  math.compile(graphExpression);
const data = d3.range(-10, 10.05, 0.1)
  .map(x => ({
    x,
    y: compiled.evaluate({ x })
  }));
```

📎 **math.compile:** https://mathjs.org/docs/reference/functions/compile.html

Going Further: Multi-Line Chart

EXTENSION

The same D3 concepts — just applied multiple times.

What Is It?

A multi-line chart draws several lines on the same graph — one per data series. Each line uses the same scales and axes, but gets its own color and dataset.

The D3 gallery has a live interactive example: hover over the chart and the closest line and data point are highlighted automatically.

Connection To Our Project

- Same d3.line() generator we already use
- Same scaleLinear scales for x and y
- Same axes — nothing new needed there
- Data binding maps each series to a path
- Could extend our calculator to graph sin(x) and cos(x) simultaneously

What's new vs. our calculator: Loop over multiple expressions · Assign a unique color per line · Add a color legend

📎 **Live example:** https://observablehq.com/@d3/multi-line-chart/2

Multi-Line Chart: How It Works

TUTORIAL

Three steps beyond a single-line chart.

1 Define a color scale — one color per series

```
const color = d3.scaleOrdinal()
  .domain(expressions)          // ["sin(x)", "cos(x)", "x^2"]
  .range(["#00B4D8", "#34D399", "#FBBF24"]); // one color per line
```

2 Loop over expressions — compile, evaluate, draw one path each

```
expressions.forEach(expr => {
  const data = d3.range(-10, 10.05, 0.1)
    .map(x => ({ x, y: math.compile(expr).evaluate({ x }) }));
  chart.append("path").datum(data)
    .attr("d", line).attr("stroke", color(expr)).attr("fill", "none");
});
```

3 Add a legend so viewers know which line is which

📎 Live example:
https://observablehq.com/@d3/multi-line-chart/2

Live Demo

DEMO TIME

01	Normal Calculation	Type 2+3*4 → press = → expect 14
02	Graph a Parabola	Clear → type x^2-4 → press Graph
03	Hover Interaction	Move over graph points — watch tooltip and crosshair
04	Zoom & Pan	Scroll or drag to explore — double-click or Reset Zoom to reset
05	Another Expression	Clear → type sin(x) → press Graph

What We Covered

Selections

d3.select() chose the SVG element to work with

Transitions

Animation drew the graph line on screen

Scales & Axes

Mapped math coords to pixels; rendered tick marks & grid

Events & Zoom

Tooltip, crosshair, pan & zoom made it interactive

Line & Area

Shape generators drew the function curve and fill

Math.js

Handled expression parsing so D3 had data to visualize

Data Binding

selectAll / data / join created hover circles from data

"The calculator stores an expression → Math.js evaluates/compiles it → D3 turns the data points into an animated, interactive SVG graph."

Official Documentation

REFERENCE

D3.js

Main D3 site: https://d3js.org/

Getting started: https://d3js.org/getting-started

Selections: https://d3js.org/d3-selection

Selecting elements: https://d3js.org/d3-selection/selecting

Modifying elements: https://d3js.org/d3-selection/modifying

Linear scales: https://d3js.org/d3-scale/linear

Axes: https://d3js.org/d3-axis

Lines: https://d3js.org/d3-shape/line

Areas: https://d3js.org/d3-shape/area

Transitions: https://d3js.org/d3-transition

Zoom behavior: https://d3js.org/d3-zoom

Math.js

Main Math.js docs: https://mathjs.org/docs/

Expression parsing & eval: https://mathjs.org/docs/expressions/parsing.html

math.evaluate: https://mathjs.org/docs/reference/functions/evaluate.html

math.compile: https://mathjs.org/docs/reference/functions/compile.html

math.parse: https://mathjs.org/docs/reference/functions/parse.html